

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA05	Course Title:	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	50 (5 Questions × 10 Mark)
CO1 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison.(BL4).</i>		
Student Name:	T.V.SAI JAGADEESH	Reg. No.:	192472146
Section / Batch:	SLOT-D/2024	Date:	7 Aug 2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10

Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
GRAND TOTAL:		50 Marks	

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

Aim:

To compare Harris Corner Detection and SIFT for feature extraction and matching.

Tools Required:

- Python
- OpenCV library
- Input images

Theory:

Harris detects corners using intensity variation, while SIFT extracts scale and rotation invariant keypoints using Difference of Gaussian.

Algorithm:

1. Read the input image and convert it into grayscale format.
2. Apply Harris corner detection by computing image gradients and corner response.
3. Mark detected corner points on the image.
4. Apply SIFT feature detector to extract keypoints and descriptors.
5. Draw keypoints on the image for visualization.
6. Compare both methods based on number and quality of detected features.

Code:

```
import cv2

img = cv2.imread('image.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

harris = cv2.cornerHarris(gray, 2, 3, 0.04)

img[harris > 0.01*harris.max()] = [0,0,255]

sift = cv2.SIFT_create()

kp, des = sift.detectAndCompute(gray, None)
```

```
img_sift = cv2.drawKeypoints(gray, kp, None)
```

```
cv2.imshow("Harris", img)
```

```
cv2.imshow("SIFT", img_sift)
```

```
cv2.waitKey(0)
```

Analysis:

- Harris detects only corners
- SIFT detects more stable features
- SIFT is better for matching

Ouptut:

```
at1.py > ...
1  import cv2
2  img = cv2.imread('image.png')
3  gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4  harris = cv2.cornerHarris(gray, 2, 3, 0.04)
5  img[harris > 0.01*harris.max()] = [0,0,255]
6  sift = cv2.SIFT_create()
7  kp, des = sift.detectAndCompute(gray, None)
8  img_sift = cv2.drawKeypoints(gray, kp, None)
9  cv2.imshow("Harris", img)
10 cv2.imshow("SIFT", img_sift)
11 cv2.waitKey(0)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

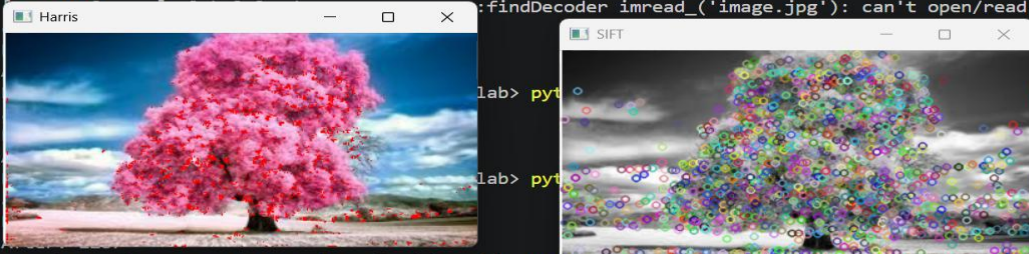
PS C:\Users\T.V.SAI JAGADEESH\Downloads\cv lab> python -u "c:\Users\T.V.SAI JAGADEESH\Downloads\cv lab\at2.py"

findDecoder imread_('image.jpg'): can't open/read file:

lab> pyt

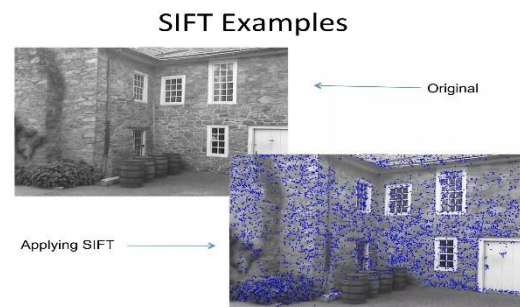
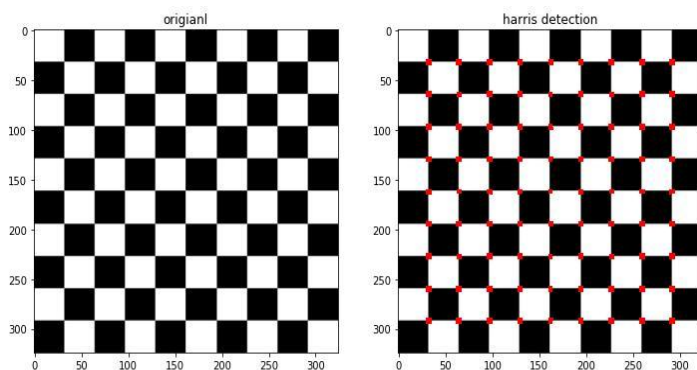
lab> pyt

PS C:\Users\T.V.SAI JAGADEESH\Downloads\cv lab> python -u "c:\Users\T.V.SAI JAGADEESH\Downloads\cv lab\at1.py"



Conclusion:

SIFT is more robust and suitable for feature matching applications.



2. Image Enhancement for Feature Detection:

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

Aim:

To evaluate the effect of image enhancement on feature detection.

Tools Required:

- Python
- OpenCV
- Sample images

Theory:

Image enhancement improves contrast and visibility, which increases the number of detectable features.

Algorithm:

1. Load the input image in grayscale format.
2. Apply histogram equalization to enhance contrast.
3. Perform feature detection on the original image.

4. Perform feature detection on the enhanced image.
5. Count and compare the number of detected features.
6. Analyze improvement in feature visibility and clarity.

Code:

```
import cv2

img = cv2.imread('image.jpg',0)

equalized = cv2.equalizeHist(img)

sift = cv2.SIFT_create()

kp1 = sift.detect(img, None)

kp2 = sift.detect(equalized, None)

print("Before:", len(kp1))

print("After:", len(kp2))
```

Analysis:

- Enhanced image produces more keypoints
- Features become clearer

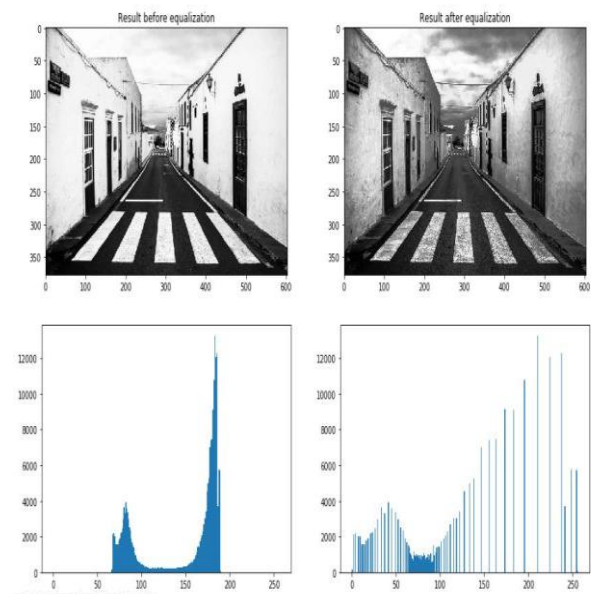
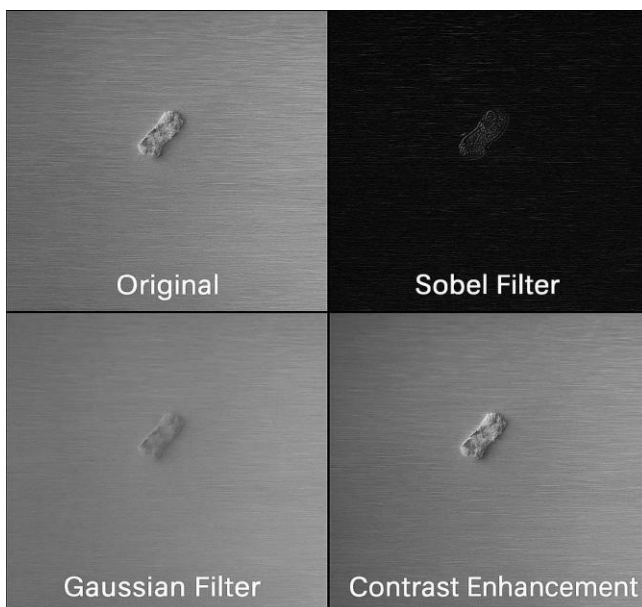
Output:

```
5.py at4.py at3.py at2.py exp-22 lapsharpe(positive).py exp-
at2.py > ...
1 import cv2
2 img = cv2.imread('image.png',0)
3 equalized = cv2.equalizeHist(img)
4 sift = cv2.SIFT_create()
5 kp1 = sift.detect(img, None)
6 kp2 = sift.detect(equalized, None)
7 print("Before:", len(kp1))
8 print("After:", len(kp2))
9 img_kp1 = cv2.drawKeypoints(img, kp1, None)
10 img_kp2 = cv2.drawKeypoints(equalized, kp2, None)
11 cv2.imshow("Original Image Keypoints", img_kp1)
12 cv2.imshow("Equalized Image Keypoints", img_kp2)
13 cv2.waitKey(0)
14 cv2.destroyAllWindows()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\T.V.SAI JAGADEESH\Downloads\cv lab> python -u "c:\Users\T.V.SAI JAGADEESH\Downloads\
ab\at2.py"
Before: 1152
After: 1287
```

Conclusion:

Image enhancement improves feature detection accuracy significantly.



3.Multi-Scale Feature Detection Analysis:

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

Aim:

To study feature detection at different image scales.

Tools Required:

- Python
- OpenCV
- Sample images

Theory:

Scale-space representation allows detection of features at multiple resolutions, ensuring invariance to size changes.

Algorithm:

1. Read the input image and convert it to grayscale.
2. Resize the image into multiple scales (small, medium, large).
3. Apply SIFT feature detection on each scaled image.

4. Count the number of detected keypoints at each scale.
5. Compare feature stability and distribution across scales.
6. Analyze the effect of scaling on detection performance.

Code:

```
import cv2

img = cv2.imread('image.jpg',0)

scales = [0.5, 1.0, 1.5]

sift = cv2.SIFT_create()

for s in scales:

    resized = cv2.resize(img, None, fx=s, fy=s)

    kp = sift.detect(resized, None)

    print(f"Scale {s}: {len(kp)} keypoints")
```

Analysis:

- Larger scale → more features
- Smaller scale → fewer details

- SIFT remains stable

```

at3.py > ...
3  img = cv2.imread('image.png',0)
4  scales = [0.5, 1.0, 1.5]
5
6  sift = cv2.SIFT_create()
7
8  for s in scales:
9      resized = cv2.resize(img, None, fx=s, fy=s)
10     kp = sift.detect(resized, None)
11     print(f"Scale {s}: {len(kp)} keypoints")
12 img_kp = cv2.drawKeypoints(resized, kp, None)
13 cv2.imshow("Keypoints", img_kp)
14 cv2.waitKey(0)
15 cv2.destroyAllWindows()

```

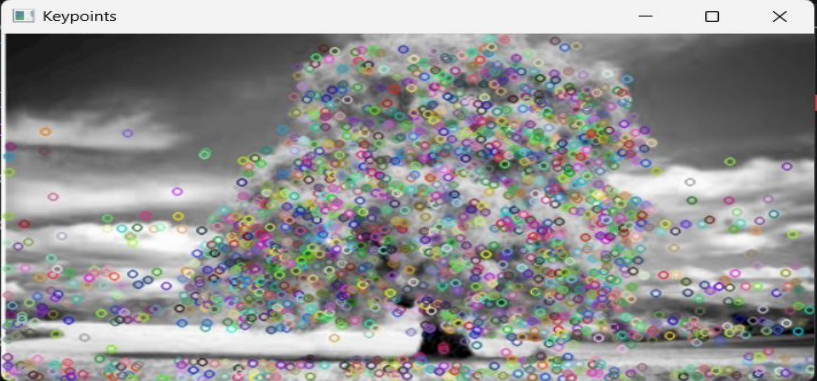
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS C:\Users\python -u "c:\Users\T.V.SAI JAGADEESH\Downloads\ab\at4.py"
cv2.error: OpenCV(4.13.0) D:\a\opencv\h.cpp:618: error: (-215:Assertion fail

● PS C:\Users\T.V.SAI JAGADEESH\Downloads\ab\at4.py"
● PS C:\Users\T.V.SAI JAGADEESH\Downloads\ab\at3.py"
Scale 0.5: 273 keypoints
Scale 1.0: 1152 keypoints
Scale 1.5: 1863 keypoints
❖ PS C:\Users\T.V.SAI JAGADEESH\Downloads\ab\at3.py"
Scale 0.5: 273 keypoints
Scale 1.0: 1152 keypoints
Scale 1.5: 1863 keypoints

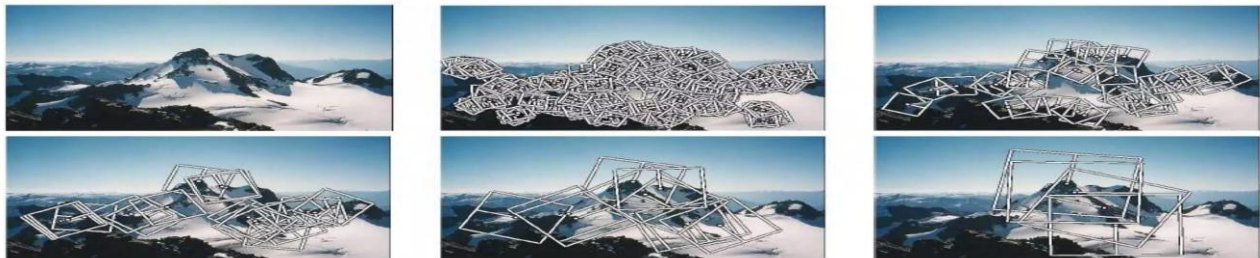
```



Conclusion:

Multi-scale detection improves robustness in real-world applications.

Detections at multiple scales



4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

Aim:

To compare Mean, Gaussian, and Median filters.

Tools Required:

- Python
- OpenCV
- Noisy images

Theory:

Filtering techniques reduce noise while preserving important features.

Algorithm:

1. Load the input noisy image.
2. Apply Mean filter to smooth the image.
3. Apply Gaussian filter for weighted smoothing.

4. Apply Median filter to remove impulse noise.
5. Display all filtered outputs.
6. Compare noise removal effectiveness and edge preservation.

Code:

```
import cv2

img = cv2.imread('image.jpg')

mean = cv2.blur(img,(3,3))

gaussian = cv2.GaussianBlur(img,(3,3),0)

median = cv2.medianBlur(img,3)

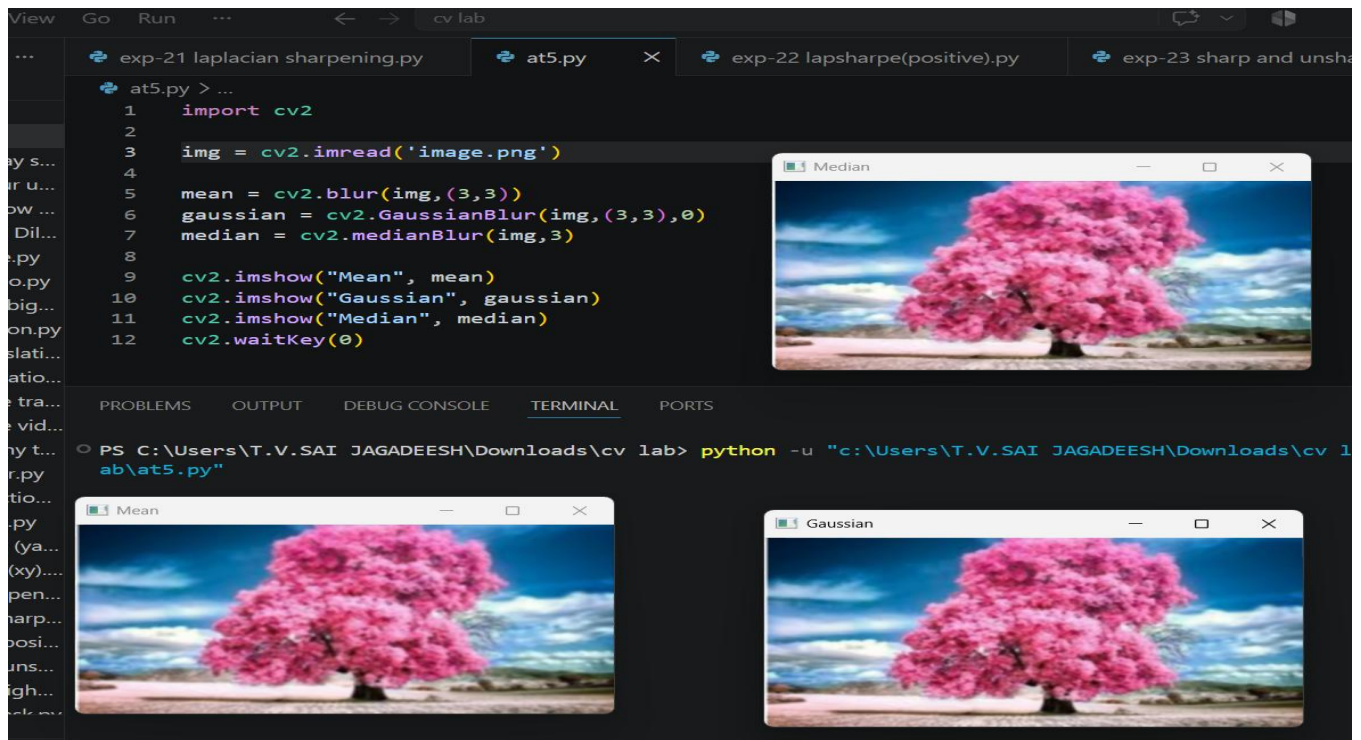
cv2.imshow("Mean", mean)

cv2.imshow("Gaussian", gaussian)

cv2.imshow("Median", median)

cv2.waitKey(0)
```

Output:



Conclusion:

Median filter is most effective for impulse noise removal.



Original Image



Gaussian blur with sigma 5



Median Blur Filter

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Aim:

To develop a complete object detection workflow.

Tools Required:

- Python
- OpenCV
- Test images

Theory:

Object detection combines preprocessing, edge detection, feature extraction, and shape detection.

Algorithm:

1. Read the input image and convert to grayscale.
2. Apply Gaussian filter to remove noise.
3. Perform edge detection using Canny operator.
4. Extract important features from edges.
5. Identify object boundaries or shapes.
6. Display and analyze detected objects.

Code:

```
import cv2
img = cv2.imread('image.jpg',0)
blur = cv2.GaussianBlur(img,(5,5),0)
edges = cv2.Canny(blur,100,200)
cv2.imshow("Edges", edges)
```

cv2.waitKey(0)

Analysis:

- Noise reduced in preprocessing
- Edges clearly detected
- Features enable object identification

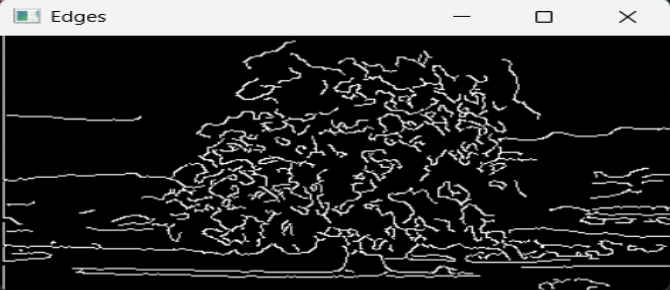
output:

```
5.py  at4.py  exp-22 lapsharpe(positive).py  exp-23 sharp and uns
```

```
at4.py > ...
1  import cv2
2
3  img = cv2.imread('image.png',0)
4
5  blur = cv2.GaussianBlur(img,(5,5),0)
6  edges = cv2.Canny(blur,100,200)
7
8  cv2.imshow("Edges", edges)
9  cv2.waitKey(0)
```

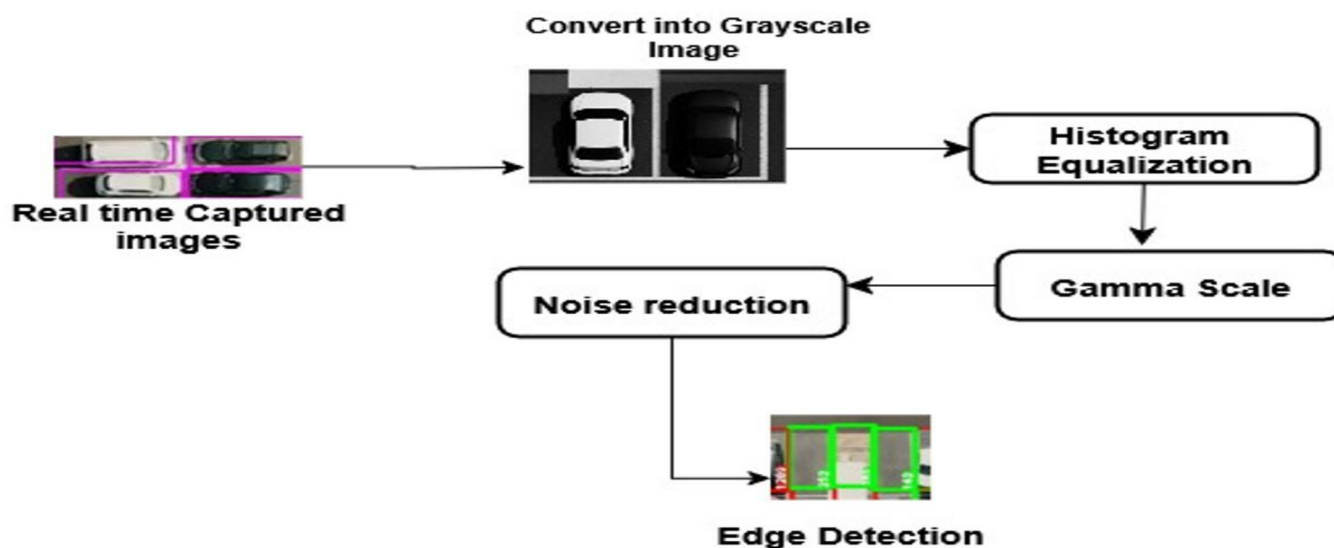
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\T.V.SAI JAGADEESH\Downloads\cv lab> python -u "c:\Users\T.V.SAI
ab\at5.py"
DEESH\Downloads\cv lab\at4.py"
::findDecoder imread_('image.jpg')
ds\cv lab\at4.py", line 5, in <mod
on\opencv-python\opencv\modules\im
!_src.empty() in function 'cv::Gau
lab> python -u "c:\Users\T.V.SAI
```



Conclusion:

Integrated workflow ensures accurate and reliable object detection.



Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, wellstructured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge

Signature: _____

Date: _____